Drone Course – Control Track, Exercises 1 and 2

Koray Közkaya

Istanbul University-Cerrahpaşa, Electrical and Electronics Engineering (3rd year)

1. Setting up the environment

I use a Windows 10 laptop, so I installed Ubuntu 22.04 with WSL2. Then I installed ROS 2 Humble (ros-humble-desktop and ros-dev-tools), cloned the course repository and installed the Aerostack2 dependencies with rosdep:

```
cd ~/project_cvar_upm_drone_course/control/drone_course_ws
rosdep install --from-paths src -y --ignore-src
colcon build --symlink-install
```

I also added the workspace setup.bash to my .bashrc so every new terminal knows the course packages.

rosdep installed all dependencies and colcon built the 4 packages of the workspace without errors.

2. What is running after launch_as2.bash

`./launch_as2.bash` starts a tmux session with the multirotor simulator (default `-p ms`). With `ros2 node list` I checked the nodes. The important ones for me were:

- /drone0/platform: the simulator, it plays the role of the real drone
- /drone0/state_estimator: publishes where the drone is (/drone0/self_localization/pose)
- /drone0/controller_manager: the PID controller that takes my reference and sends commands to the platform
- /drone0/alphanumeric_viewer and /rviz: to see the drone state as numbers and in 3D
- /gates_static_publisher_node and /gates_tracker_node: they place the gates and measure the lap times

config/initialize.bash arms the drone, puts it in offboard mode and takes off to 1 m. The control mode call is commented, so after launch the viewer shows Control Mode: UNSET.

3. First try with the command line

Before writing the node I did the whole workflow by hand in the terminal, to see how it works (I have a short video of this test too). I set the control mode with a service call on /drone0/controller/set_control_mode (control_mode 2 = POSITION, yaw_mode 1 = YAW_ANGLE, reference_frame 1 = LOCAL_ENU_FRAME), then I sent position references with ros2 topic pub on /drone0/motion_reference/pose and checked the result with ros2 topic echo /drone0/self_localization/pose. I moved the drone through the gates one by one this way.

A few things I learned here:

- I found the message fields with ros2 interface show geometry_msgs/msg/PoseStamped instead of guessing. orientation w must be 1.0, because an all-zero quaternion is not a valid rotation.
- With --once the drone did not move. The message is sent before the publisher and the controller find each other, so it is lost. With -r 10 it worked.
- Without the control mode call the controller ignores the references. After restarting the simulation the mode was UNSET again.
- The gates are at x = 5 and x = 10 (config/gates_config.yaml), but the waypoints are 1 m before and after them: (4, 0), (11, 0), (11, 5), (4, 5), all at z = 2.1. This way the drone goes straight through the gates. The same path is given by the /request_path service.

4. Exercise 1.1 with a Python node

Then I did the same thing automatically in python_interface/exercise_1/exercise_1_1.py. First I uncommented the CONTROL MODE SERVICE CALL in config/initialize.bash, so the mode is set to POSITION when the simulation starts. Then I completed the TODOs in the template.

Subscriber for the estimated pose. I used the best effort QoS because the pose comes many times per second and losing one message is not a problem:

```
self.pose_sub = self.create_subscription(
    PoseStamped,
    '/drone0/self_localization/pose',
    self.state_pose_callback,
    best_effort_qos,
)
```

Publisher for the position reference, with reliable QoS because I do not want to lose a command:

```
self.pose_pub = self.create_publisher(
    PoseStamped,
    '/drone0/motion_reference/pose',
    reliable_qos,
)
```

In run(), which is called by the timer every 0.1 s (10 Hz, same as -r 10 in the terminal), I publish the reference:

```
self.pose_pub.publish(position_msg)
```

When the distance to the current waypoint is less than 0.2 m the node goes to the next one. The template had self.path_idx = 1, which would stay at the second point forever, so I changed it to increase the index. I also check the length of the path, because after the last point self.path[4] does not exist and the node would crash with IndexError. At the last point the drone just stays there:

```
if position_error < 0.2:
    if self.path_idx < len(self.path) - 1:
        self.path_idx += 1
        self.get_logger().info(f'Waypoint reached, going to point
{self.path_idx}')
    else:
        self.get_logger().info('Circuit completed',
throttle_duration_sec=2.0)
```

To run it I start the simulation with ./launch_as2.bash and in a second terminal:

```
cd python_interface/exercise_1
python3 exercise_1_1.py
```

The drone passed all four gates by itself and the node printed Circuit completed at the end. I recorded the video of this run for the submission.

5. Exercise 1.2 – path from a service and control mode from the node

In 1.1 the path was written by hand in the code and the control mode was set by initialize.bash. In 1.2 the node does both by itself, so first I commented the CONTROL MODE SERVICE CALL in config/initialize.bash again. After launch the viewer shows UNSET until my node runs.

The template already had a service client for the control mode, so I used it as an example. I created a second client for the path service with the same pattern, only the type and the name are different:

```
self.path_request_client = self.create_client(
    RequestPath, '/request_path', callback_group=self.callback_group
)
```

In run(), if the path is still empty, the node checks that the service is ready, sends an empty request and keeps the list from the response. The request of RequestPath has no fields, and the response has one field, path, which is a list of Points (I saw this with ros2 interface show drone_course_msgs/srv/RequestPath).

```
if not len(self.path) > 0:
    if not self.path_request_client.service_is_ready():
        self.get_logger().warn('Path service is not available',
throttle_duration_sec=1.0)
        return
    req = RequestPath.Request()
    response = self.path_request_client.call(req)
    self.path = list(response.path)
    if len(self.path) == 0:
        return
```

The return is important. If the service is not ready yet, the node tries again in the next timer call (0.1 s later). It also protects the last line of the template, self.path_idx = (self.path_idx + 1) % len(self.path), because with an empty path this would be a division by zero. Because of the modulo, after the last point the index goes back to 0, so in 1.2 the drone keeps flying the circuit.

The call is synchronous inside the timer callback. This works because the node uses a MultiThreadedExecutor and a ReentrantCallbackGroup, so the response can be processed in another thread while the timer is waiting.

When I ran python3 exercise_1_2.py the node printed the four points (4, 0, 2.1), (11, 0, 2.1), (11, 5, 2.1), (4, 5, 2.1), then Control mode set successfully, and the viewer changed from UNSET to POSITION. The drone flew the circuit again and again. I recorded a video of this run too.

One problem I had: once the drone fell and the node kept printing Control mode service is not available. The controller was not running after the restart. After a clean restart (./stop.bash, checking with tmux ls that no old session was left, then ./launch_as2.bash) and checking with ros2 service list that /drone0/controller/set_control_mode exists before starting the node, it worked.

6. Exercise 2 – speed control with my own PID

First I tried speed mode from the terminal (control_mode 3 = SPEED, yaw_mode 2 = YAW_SPEED) and sent a TwistStamped with x = 0.5 to /drone0/motion_reference/twist. The drone went forward slowly and did not stop, even after the gates, until I sent zero speed. So in speed mode the drone does not care where it is, and I need a closed loop to stop it at the waypoints.

In exercise_2.py the TODO was to compute the speed commands from the position error. I wrote a PID for each axis (x, y, z):

$v = KP \cdot e + KI \cdot \int e\,dt + KD \cdot de/dt$

```
p_term = KP * e
self.integral[i] = clamp(self.integral[i] + e * dt, MAX_INTEGRAL)
i_term = KI * self.integral[i]
d_term = KD * (e - self.prev_error[i]) / dt
commands[i] = p_term + i_term + d_term
```

I also added a speed limit, a limit for the integral (anti-windup), a reset of the PID when the drone goes to the next waypoint, no derivative in the first step (otherwise the first command is very big) and a zero speed command when the node is closed, because in speed mode the drone keeps the last speed. I removed import future from the template because it is not used.

It did not work at the beginning. First I wrote KD:0.1 instead of KD = 0.1. Python does not give an error for this line, it reads it as a type annotation, so KD was never defined and the node crashed later with NameError. Then the node got stuck and printed Calling control mode service... many times. I could not find the reason myself, so I used an AI assistant for help in this exercise. The problem was that the timer runs at 100 Hz in a ReentrantCallbackGroup, so new run() calls started while the first one was still waiting for the service response, and in the end no thread was free to process the response (deadlock). The fix was to put the timer in a MutuallyExclusiveCallbackGroup and keep the service clients in the reentrant group.

Then the tuning. With KP = 1.2, KI = 0.02 and KD = 0.1 the drone shook left and right. My explanation is that the pose comes slower than my 100 Hz loop, so the difference of the error is zero in some steps and big in others, and the D term jumps. With KD = 0 the flight became smooth. In the end I used KP = 1.0, KI = 0.02, KD = 0 and a horizontal speed limit of 1.5 m/s. The drone still brakes a bit at each waypoint and moves a little left and right in the corners, because the reference changes suddenly by 90°. An acceleration limit or using the measured speed for the D term could make it better.

7. What I understood

The main thing is the closed loop. My node only says where the drone should be. The state estimator says where it is, the controller computes the error (reference minus position) and the PID turns it into commands for the platform. When the error is close to zero the drone stops at the reference. It is the same feedback structure we draw in our System Modelling course, but here I could see it working.

I also understood the difference between a topic (continuous data, no answer) and a service (one request, one response), and that every terminal command I used has a direct equivalent in the node: topic echo is the subscriber, topic pub is the publisher and -r 10 is the timer.

Exercise 2 showed me what is inside the controller I used in Exercise 1. It is also a PID (config/pid_speed_controller.yaml has kp = 1, ki = 0.01, kd = 0), and the hard part is not the formula but choosing the gains.

8. Documentation I used

- The course repository: control/docs/INSTALLATION.md for the setup and control/exercises/Exercise1.md for the steps of the exercise
- The config files of the course (config/initialize.bash, config/tmuxinator.yaml, config/gates_config.yaml) and config/gates_static_publisher.py to see how the path is generated
- ROS 2 Humble documentation (docs.ros.org/en/humble), mainly the tutorials about nodes, topics and services, and the page about quaternion fundamentals
- Aerostack2 documentation (aerostack2.github.io) for the control modes and the meaning of the indices in SetControlMode
- The ROS 2 command line itself: ros2 topic info, ros2 interface show and ros2 service list to find the message types and fields